



FTS-Diffusion: Generative Learning for Financial Time Series

Deli Lin Eric Shao Lapo Linossi Tom Haidinger

Machine Learning in Finance and Complex Systems

18 May 2026



Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

The problem: financial time series are hard to generate

Empirical stylized facts

- Heavy-tailed returns; rare but consequential crises.
- Volatility clustering, long memory in $|r_t|$.
- Scale-invariant patterns at irregular timescales.
- Single realised history (no resampling).

Why standard generators fail

Fixed-window models segment in **calendar time**, not **pattern time**. The same shape at twice the duration becomes a different sample.

The paper's contribution

FTS-Diffusion (Yu et al., ICLR 2024): a three-module pipeline.

- **SISC**: Scale-Invariant Subsequence Clustering, discovers a finite dictionary of variable-length motifs using DTW.
- **PGM**: Pattern Generation Module, a conditional diffusion network that synthesises new latent motifs.
- **PEM**: Pattern Evolution Module, a Markov chain over latent states (p_m, α_m, β_m) steering long-horizon dynamics.

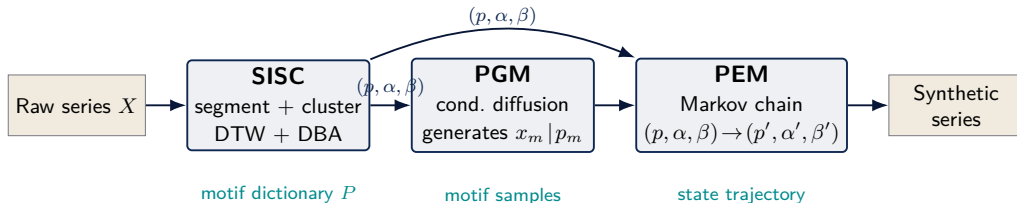
Our goal

Replicate the pipeline, stress-test it on three assets (S&P 500, GOOG, ZC=F), and isolate the methodological choices that matter.

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

FTS-Diffusion pipeline



State variables

- p : pattern identity (cluster label).
- α : duration scale factor (length stretch).
- β : magnitude scale factor (amplitude).

DTW: align in time before measuring distance.
Two segments stay close even if one is stretched.

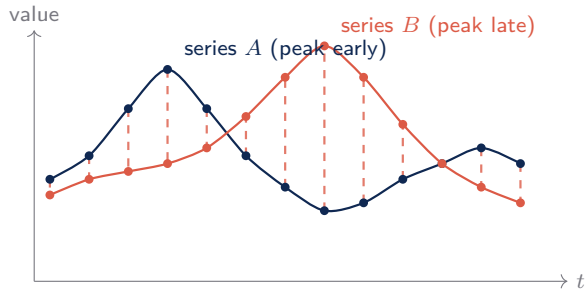
Why K-means++ for centroid init?

- K-means++ seeds far-apart centroids, more stable clusters.
- Plain K-means can collapse two centroids onto the same regime.

Outline

1. Motivation
2. Architecture overview
3. **SISC: three animated steps**
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

DTW: Align in time before measuring distance

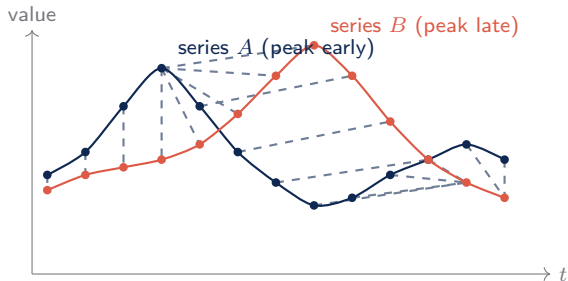


Pointwise: Strict $t_i \leftrightarrow t_i$ matching penalizes shifts

Euclidean Distance rigidly compares values at identical time indices, ignoring phase shifts entirely.

- **The Solution:** DTW permits non-linear mapping, concluding *“same shape, just stretched in time”* → small distance.

DTW: Align in time before measuring distance



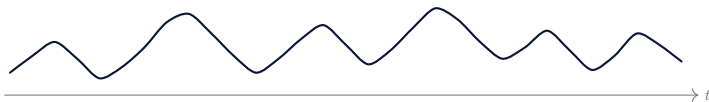
DTW: One-to-many matching aligns morphological features

DTW calculates the optimal monotone alignment of the two series, then measures distance along that warped path.

- **The Solution:** DTW permits non-linear mapping, concluding *“same shape, just stretched in time”* → small distance.
- **Application:** This property is exactly what we need for financial motifs that recur with varying durations.

SISC, Step 1: Initializing centroids (K-means++ for DTW)

observed series X (candidate windows shaded in coral)



centroid slots

$K = 4,$

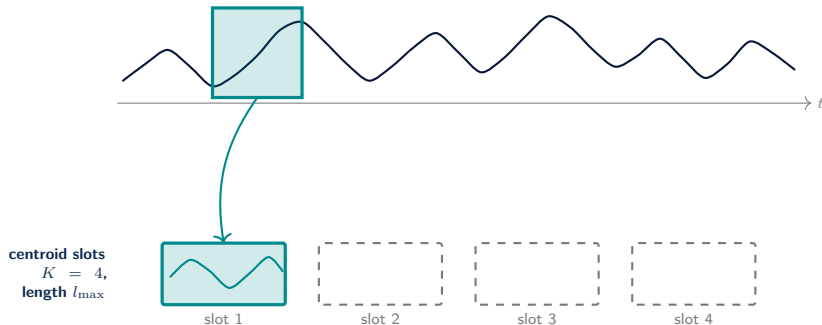
length $l_{\max}$



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 0: $K = 4$ centroids of fixed length $l_{\max}$ to be chosen.

SISC, Step 1: Initializing centroids (K-means++ for DTW)

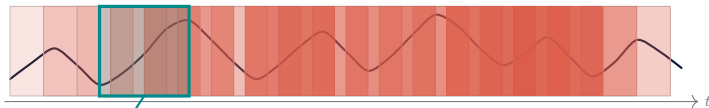
observed series X (candidate windows shaded in coral)



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 1: p_1 sampled uniformly from all length- $l_{\max}$ windows.

SISC, Step 1: Initializing centroids (K-means++ for DTW)

observed series X (candidate windows shaded in coral)



centroid slots
 $K = 4$,
length $l_{\max}$



slot 1



slot 2



slot 3

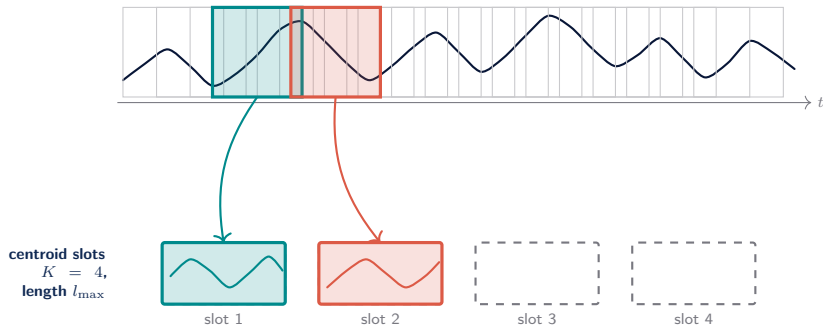


slot 4

Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 2: candidate windows weighted by distance to p_1 (coral = far).

SISC, Step 1: Initializing centroids (K-means++ for DTW)

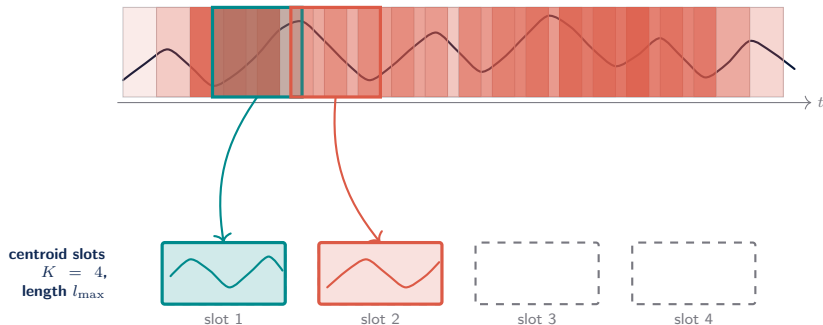
observed series X (candidate windows shaded in coral)



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 3: p_2 sampled with prob. $\propto \text{DTW}(\cdot, p_1)$.

SISC, Step 1: Initializing centroids (K-means++ for DTW)

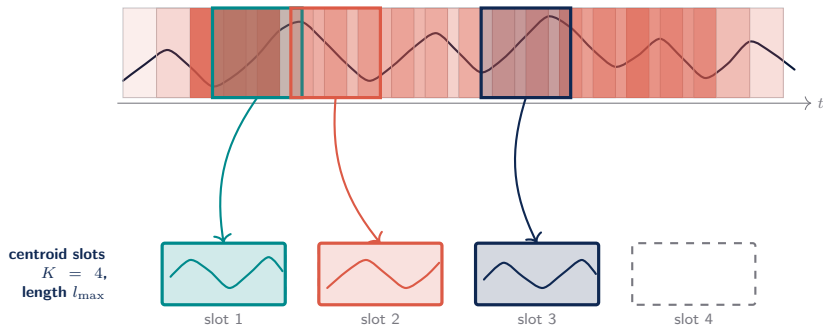
observed series X (candidate windows shaded in coral)



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 4: recompute weights against $\{p_1, p_2\}$.

SISC, Step 1: Initializing centroids (K-means++ for DTW)

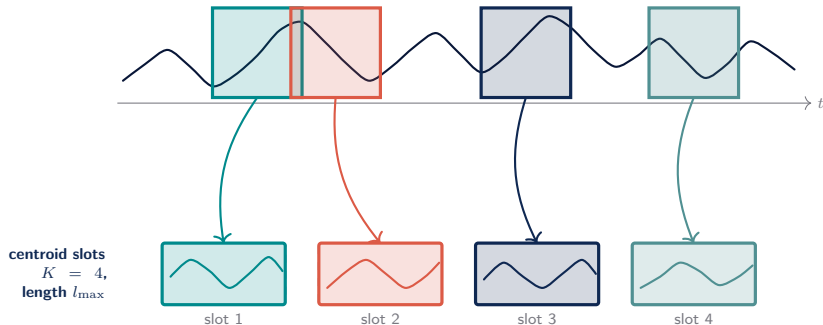
observed series X (candidate windows shaded in coral)



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 5: p_3 sampled with prob. $\propto \min_{k' \leq 2} \text{DTW}(\cdot, p_{k'})$.

SISC, Step 1: Initializing centroids (K-means++ for DTW)

observed series X (candidate windows shaded in coral)

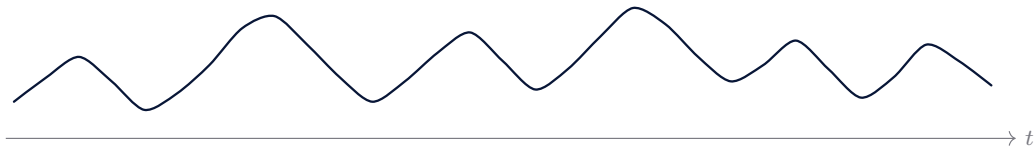


Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 6: p_4 chosen; initialisation complete.

SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

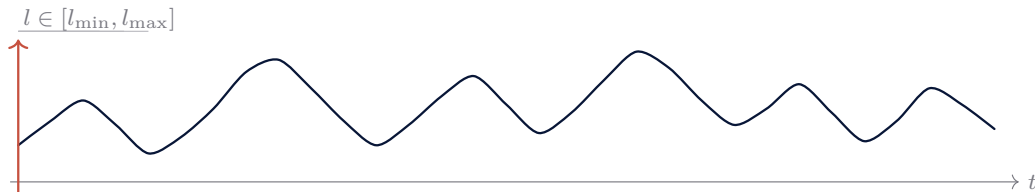
$$P = \{p_1, \dots, p_4\}$$



SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$



$$K \cdot (l_{\max} - l_{\min} + 1) = 168 \text{ DTW evals per step}$$

SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$



p_1



p_2



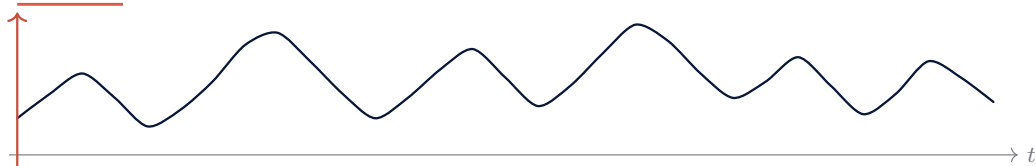
p_3



p_4

$$l^* = \text{len}(x_m) / \text{len}(\text{centroid}(p_m)), \text{ dimensionless ratio}$$

$$l^* = 1.3$$

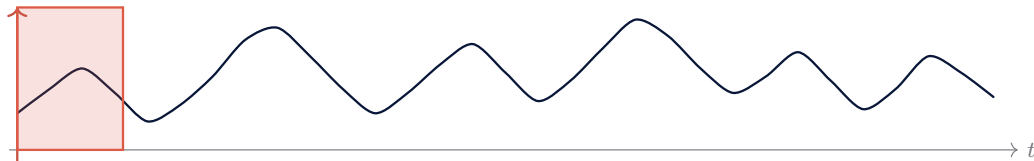


$$K \cdot (l_{\max} - l_{\min} + 1) = 168 \text{ DTW evals per step}$$

SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$

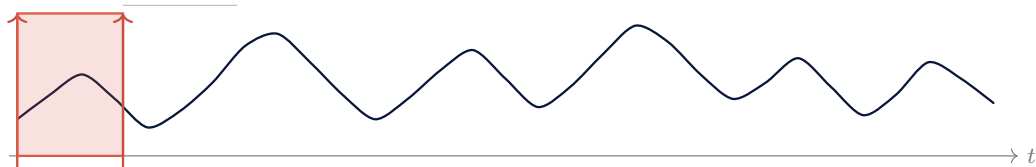


greedy: emit best (l^*, k^*) , advance

SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$

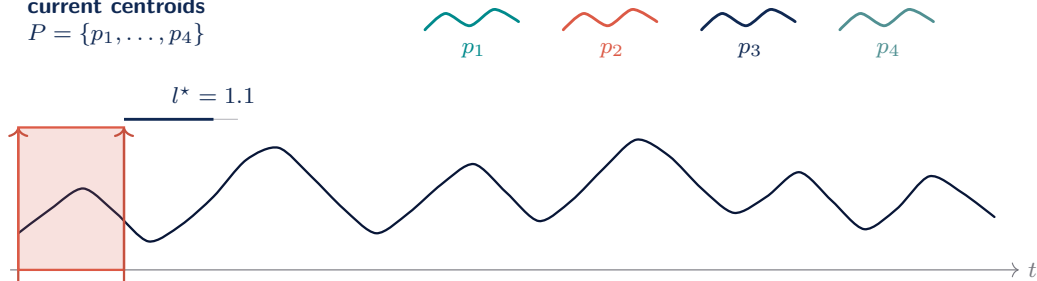


greedy: emit best (l^*, k^*) , advance

SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$

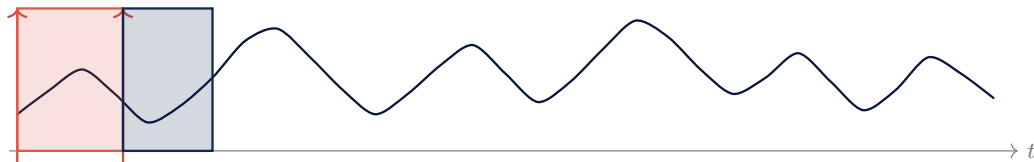


greedy: emit best (l^*, k^*) , advance

SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$



greedy: emit best (l^*, k^*) , advance

SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$



p_1



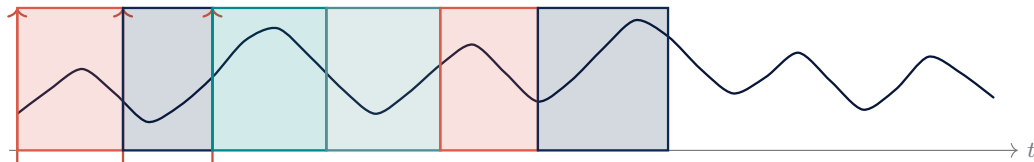
p_2



p_3



p_4



variable-length segmentation continues

SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$



p_1



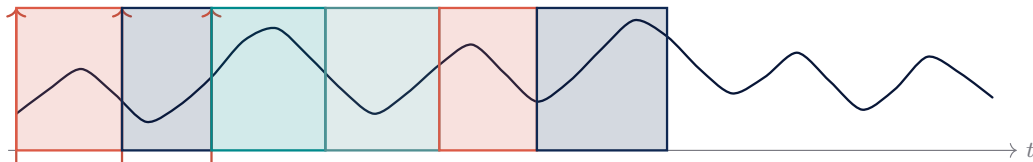
p_2



p_3

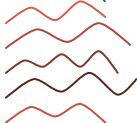


p_4



variable-length segmentation continues

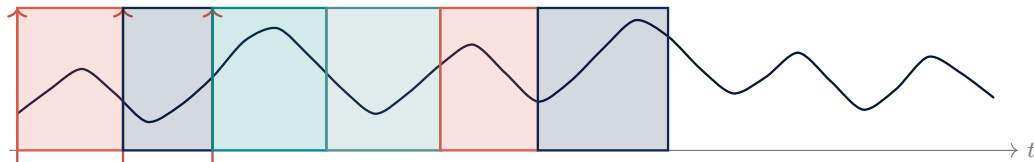
cluster C_k (members)



SISC, Step 2: Greedy segmentation + DBA centroid update

current centroids

$$P = \{p_1, \dots, p_4\}$$



variable-length segmentation continues

cluster C_k (members)

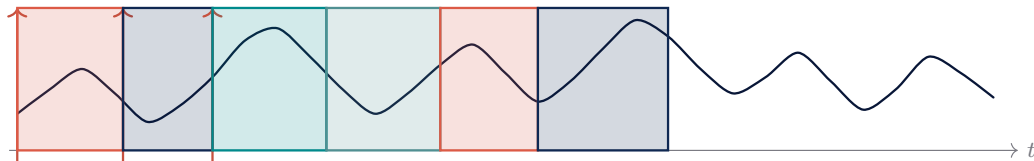
DTW alignments



SISC, Step 2: Greedy segmentation + DBA centroid update

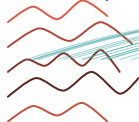
current centroids

$$P = \{p_1, \dots, p_4\}$$



variable-length segmentation continues

cluster C_k (members)



DTW alignments

centroid index

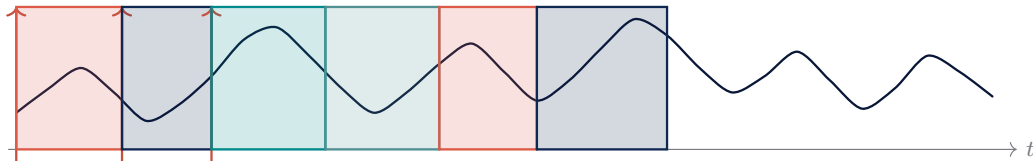
new centroid p_k^{new} (DBA)



SISC, Step 2: Greedy segmentation + DBA centroid update

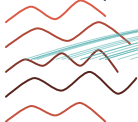
current centroids

$$P = \{p_1, \dots, p_4\}$$



variable-length segmentation continues

cluster C_k (members)



DTW alignments

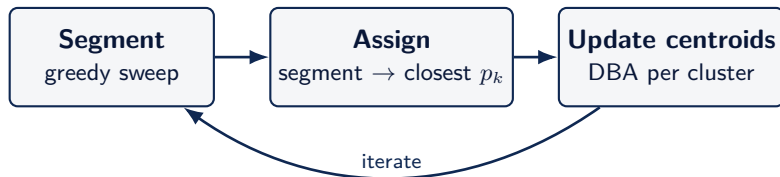
new centroid p_k^{new} (DBA)



DBA (DTW Barycenter Averaging): centroid is the **average over aligned values**, not pointwise. It produces a barycenter of variable-length sequences in DTW space.

$$p_k^{\text{new}} = \arg \min_p \sum_{x \in C_k} \text{DTW}(x, p)^2.$$

SISC, Step 3: Iterate until convergence



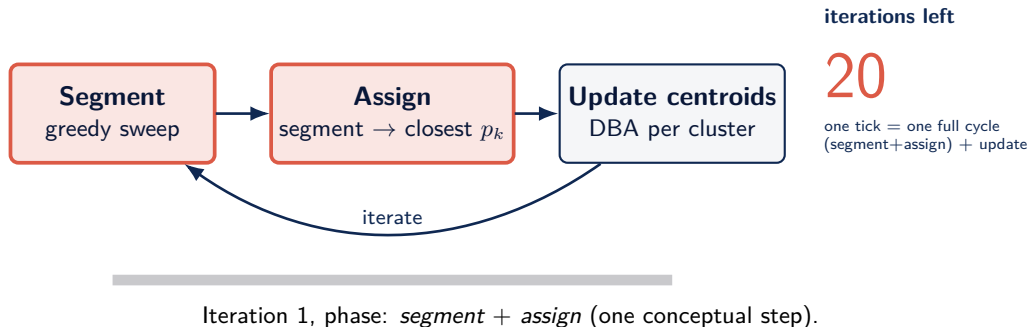
iterations left

20

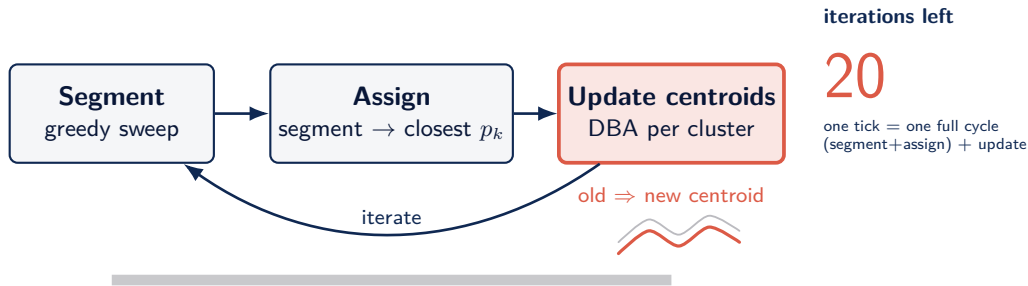
one tick = one full cycle
(segment+assign) + update

Step 0: 20-iteration budget, no work yet.

SISC, Step 3: Iterate until convergence

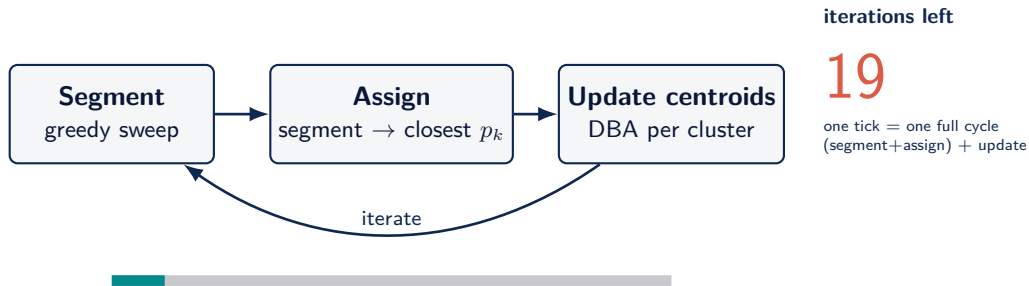


SISC, Step 3: Iterate until convergence



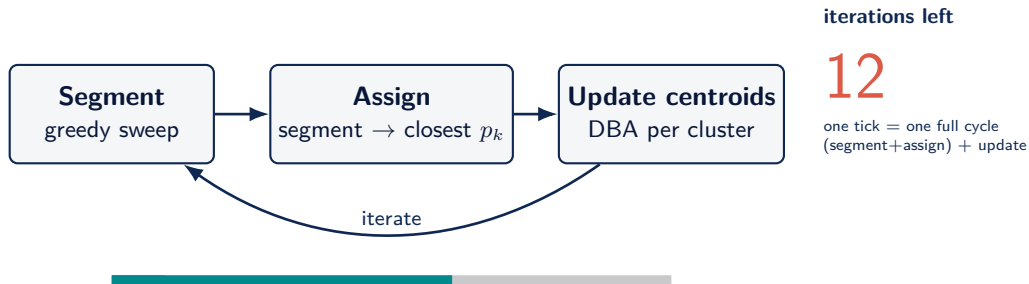
Iteration 1, phase: *centroid update* (DBA). Counter still 20.

SISC, Step 3: Iterate until convergence



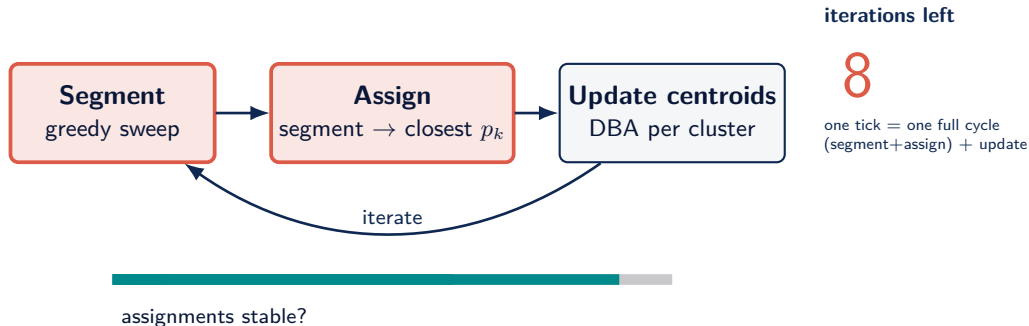
End of iteration 1: full cycle complete, budget ticks to 19.

SISC, Step 3: Iterate until convergence



Fast-forward several iterations: shifts shrink, budget reaches 12.

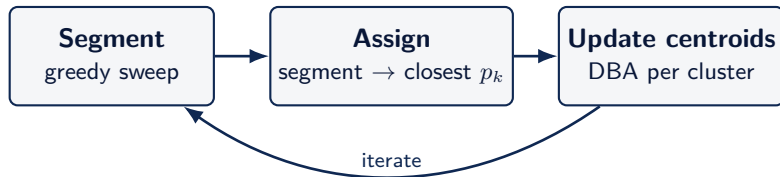
SISC, Step 3: Iterate until convergence



assignments stable?

Stability check: are assignments unchanged across cycles?

SISC, Step 3: Iterate until convergence



iterations left

8

one tick = one full cycle
(segment+assign) + update

assignments stable? ✓ **converged**

Convergence. Stops on stable assignments OR budget exhausted.

Limits of the SISC algorithm

Local minima risk

SISC is a non-convex optimisation: greedy segmentation followed by iterative centroid update. It can stall when motifs are noisy or weakly separated.

Initialization sensitivity

Bad initial centroids yield a bad final segmentation. This is exactly why we use K-means++ for the seeding step (back-reference to the pipeline slide).

Hyperparameter sensitivity

K and $[l_{\min}, l_{\max}]$ must be chosen up front. Mis-set values either over-fragment (too many tiny motifs) or under-segment (one motif swallows the series).

Inherent to any non-convex clustering procedure on time-series data: we don't take a clean segmentation for granted.

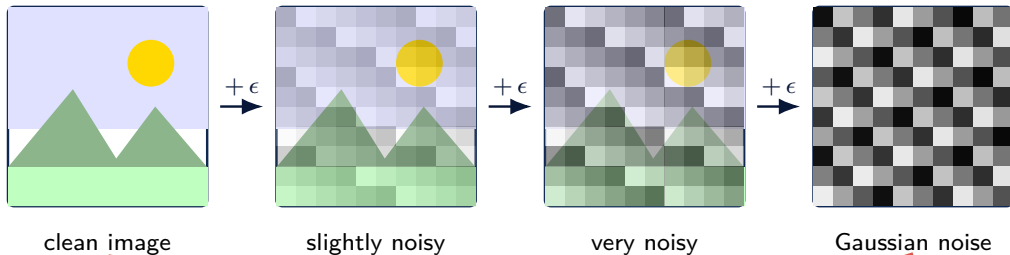
Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
- 4. Generation module (PGM)**
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

Image Diffusion

Destroy an image with noise, then learn to reverse the process

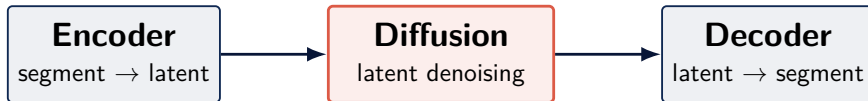
Forward diffusion



learn reverse denoising
Reverse generation

Generation module: encoder, diffusion, decoder

The PGM pipeline in its simplest form



Takeaway

$$p_{\theta}(\cdot \mid p, \alpha, \beta) \approx q(\cdot \mid p, \alpha, \beta)$$

It is essentially trying to learn the distribution function $q(\cdot \mid p, \alpha, \beta)$, where p is the pattern type, α the duration scale, and β the magnitude scale.

Scaling autoencoder

From variable-length segments to fixed-length latent sequences

Encoder

Maps a variable-length segment to a padded latent sequence:

$$x_m \longrightarrow z_m \in \mathbb{R}^{l_{\max} \times 1}.$$

Released code:

pack input lengths

→ 2-layer LSTM, hidden dim 1

→ linear projection, dim 1

Decoder

Maps the latent sequence back to a variable-length segment:

$$z_m \longrightarrow \hat{x}_m.$$

Released code:

pack latent lengths

→ 2-layer LSTM, hidden dim 1

→ linear output, dim 1

Model choice

Paper: two LSTM/GRU layers are allowed. Released code: two-layer LSTM encoder and decoder, hidden dimension 1, padded length $l_{\max}$.

Loss: reconstruction term

The full objective, highlighting the scaling autoencoder part

Pattern generation objective

$$\mathcal{L}(\theta) = \mathbb{E}_{x_m} [\|x_m - \hat{x}_m\|_2^2] + \mathbb{E}_{x'_m, i, \epsilon} [\|\epsilon_i - \epsilon_\theta(x_m^i, i, p)\|_2^2]$$



reconstruction loss

Latent residual for diffusion

The model corrupts the encoded latent sequence relative to the fixed pattern

Forward process in latent notation:

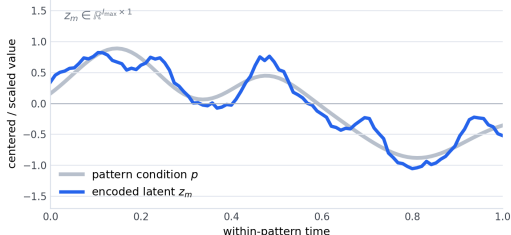
p : pattern condition/type

β : diffusion noise scale

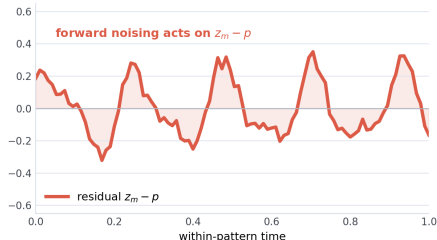
$$q(z^i | z^{i-1}, p) = \mathcal{N}\left(z^i; \sqrt{1 - \beta} (z^{i-1} - p), \beta I\right)$$

Center at p and diffuse the residual $z^{i-1} - p$.

latent sequence and condition

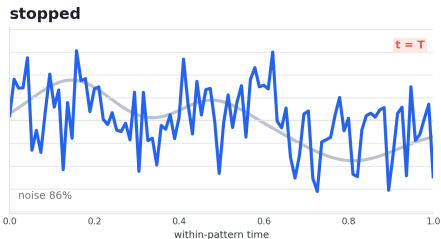
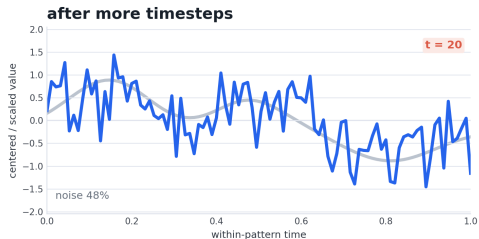
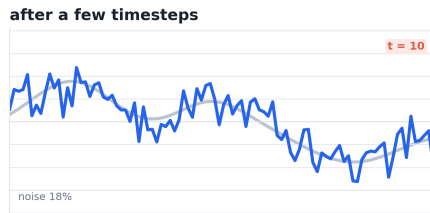
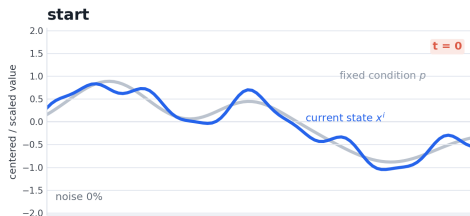


what diffusion corrupts



Pattern-conditioned diffusion: forward noising

Training corrupts the residual/latent state around the fixed pattern



Pattern-conditioned diffusion: backward denoising

Sampling learns the reverse transition, conditioned on the fixed pattern

Backward process in latent notation:

$$p_{\theta}(z^{i-1} | z^i, p) = \mathcal{N}(z^{i-1}; \mu_{\theta}(z^i, i, p), \beta I)$$

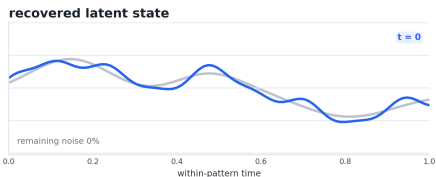
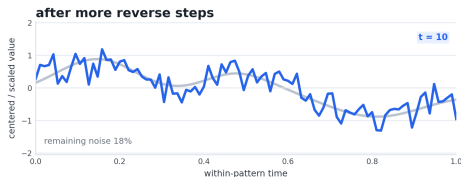
μ_{θ} : TCN2 denoiser (released code)

6 residual TCN blocks, kernel 3

channels: 48, 64, 80, 80, 64, 48

dilations: 1, 2, 4, 8, 16, 32

time embedding 32, steps 30



Deriving the diffusion loss

Train the denoiser by asking it to recover the noise that was added

1. Residual

$$r_0 = z_m - p$$

Keep p fixed and corrupt the residual.

2. Closed-form noising

$$r_i = \sqrt{\bar{\alpha}_i} r_0 + \sqrt{1 - \bar{\alpha}_i} \epsilon$$

$$\epsilon \sim \mathcal{N}(0, I), \quad \bar{\alpha}_i = \prod_{s=1}^i (1 - \beta_s).$$

3. Predict noise

$$\epsilon_{\theta}(r_i, i, p) \approx \epsilon$$

Train on the known noise sample.

Diffusion loss

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{r_0, i, \epsilon} [\|\epsilon - \epsilon_{\theta}(r_i, i, p)\|_2^2]$$

Loss: diffusion term

The full objective, highlighting the noise-prediction part

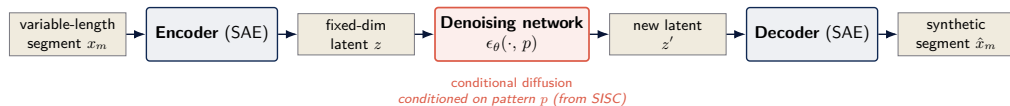
Pattern generation objective

$$\mathcal{L}(\theta) = \mathbb{E}_{x_m} [\|x_m - \hat{x}_m\|_2^2] + \mathbb{E}_{x'_m, i, \epsilon} [\|\epsilon_i - \epsilon_\theta(x_m^i, i, p)\|_2^2]$$


diffusion loss

Step 2: pattern generation

How does diffusion know what to generate? Conditioning happens in latent space.



Conditional forward step (paper §4.2):

$$q(x^i | x^{i-1}) = \mathcal{N}(x^i; \sqrt{1 - \beta} (x^{i-1} - p), \beta I).$$

Noise is added to the residual ($z - p$), so the denoiser learns to produce segments resembling the centroid of pattern p , not arbitrary signals.

- SAE encoder maps variable-length $\rightarrow$ fixed-length latent.
- Diffusion runs in the latent, conditioned on p .
- SAE decoder maps the new latent back to a segment.
- **Coherent pattern evolution** prevents learning pure noise.

Pattern-conditioned diffusion network

Learning to denoise latent pattern representations

Forward diffusion

Gradually corrupt the latent with Gaussian noise:

$$q(x^i | x^{i-1}) = \mathcal{N}(x^i; \sqrt{1 - \beta}(x^{i-1} - p), \beta I).$$

Reverse denoising

A network learns to recover the previous step, conditional on pattern p :

$$p_\theta(x^{i-1} | x^i) = \mathcal{N}(x^{i-1}; \mu_\theta(x^i, i, p), \beta I).$$

Intuition

Map noise to realistic pattern-level market segments.

Training objective: paper vs. code

The paper states an unweighted objective; the released code adds weights

Paper objective (Eq. 5)

$$\mathcal{L}(\theta) = \mathbb{E}_{x_m} [\|x_m - \hat{x}_m\|_2^2] + \mathbb{E}_{x'_m, i, \epsilon} [\|\epsilon_i - \epsilon_\theta(x_m^i, i, p)\|_2^2] .$$

Meaning

- First term: reconstruction loss for the scaling autoencoder.
- Second term: diffusion noise-prediction loss.
- The paper does **not** introduce explicit λ weights here.

Implementation note

The released code uses:

`loss_weights = [0.98, 0.01],` `scale_weight = 0.01.`

These are implementation weights, not paper notation.

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
- 5. Evolution module (PEM)**
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

Step 3: pattern evolution

Markets evolve through regimes; PEM learns the empirical transition kernel

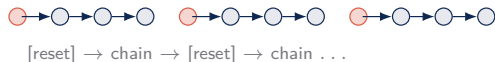
Markets evolve through regimes (bull/bear, low-vol/high-vol). The PEM learns the empirical transition kernel between the latent regimes that SISC discovered.

What the PEM does

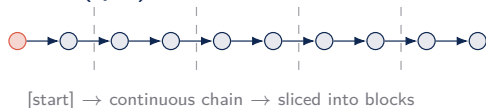
1. Governs the **temporal structure** of the synthetic series.
2. Assumes Markov dynamics: $S_{m+1} \sim Q(\cdot | S_m)$.
3. **Sensitive to the rollout protocol** (headline of the next slides).
4. Iterated **autoregressively** at sampling time: MAPE worsens as more synthetic data is fed in, errors compound.
5. **Low-dimensional**: collapses each segment to (p_m, α_m, β_m) .

Authors **reset** the chain at the start of every downstream block. We let the chain **evolve continuously** and slice it. This subtle choice flips the result for S&P 500 (we'll see how in the results section).

authors (reset)



ours (split)



Pattern evolution network

Classification for patterns, regression for scales

Transition network

$$(\hat{p}_{m+1}, \hat{\alpha}_{m+1}, \hat{\beta}_{m+1}) = \phi(p_m, \alpha_m, \beta_m).$$

Outputs

- next pattern p_{m+1} ;
- next duration scale α_{m+1} ;
- next magnitude scale β_{m+1} .

Learning tasks

- pattern: classification;
- duration: regression;
- magnitude: regression.

$$\mathcal{L}(\phi) = \mathbb{E}_{x_m} [\ell_{CE}(p_{m+1}, \hat{p}_{m+1}) + \|\alpha_{m+1} - \hat{\alpha}_{m+1}\|_2^2 + \|\beta_{m+1} - \hat{\beta}_{m+1}\|_2^2].$$

Full synthetic time-series generation

Putting the three modules together

1. Start from an initial historical segment.
2. Use the evolution network to predict $(\hat{p}_{m+1}, \hat{\alpha}_{m+1}, \hat{\beta}_{m+1})$.
3. Generate the next segment via the pattern-conditioned diffusion model.
4. Decode it into a variable-length financial segment.
5. Append the segment to the synthetic series.
6. Repeat until the desired path length is reached.

Intuition

Local motifs are produced by diffusion; global temporal structure is controlled by the evolution network.

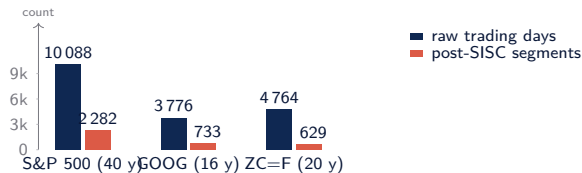
Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
- 6. Training data**
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

Training data per asset

Strong size asymmetry across the three assets

Asset	Type	Period	Years (days)	Post-SISC seg. (K)	Data pipeline
S&P 500	index	1980–2020	40 (10 088)	2 282 (14)	• Source: Yahoo Finance daily close.
GOOG	stock	2004–2020	16 (3 776)	733 (11)	• Modality: raw prices , not returns.
ZC=F	future	2000–2020	20 (4 764)	629 (11)	• FTS-Diffusion split: 80 % train / 20 % test.



- **Downstream LSTM:** carved from the 20 % test slice.

Size asymmetry

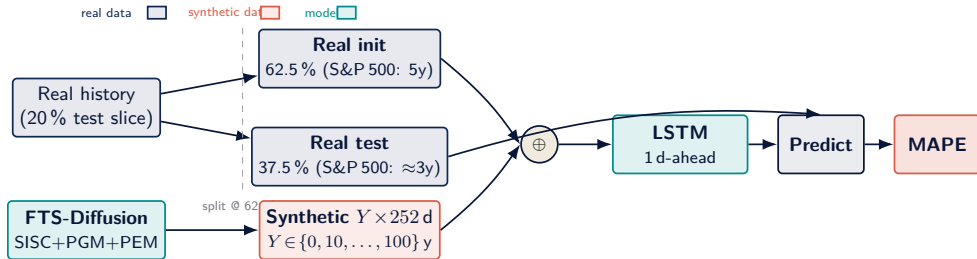
After segmentation, S&P 500 is $\approx 3\times$ GOOG and $\approx 3.6\times$ ZC=F.

Outline

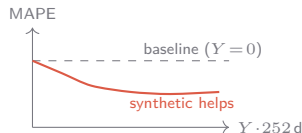
1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
- 7. TATR: the downstream benchmark**
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

TATR: Train on Augmentation, Test on Real

Does FTS-Diffusion synthetic data *help* a forecaster?



- **Train:** $\text{real_init} \oplus \text{synth}[0 : Y \cdot 252]$.
- **Test:** real held-out (never seen during training).
- **Sweep:** $Y \in \{0, 10, 20, \dots, 100\}$ years of synthetic.
- **Score:** MAPE on real test, mean over 15 seeds.



One question only

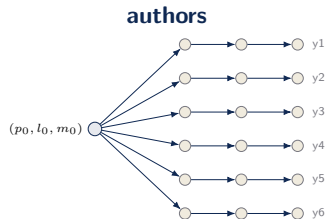
Does adding Y years of synthetic data **reduce** the LSTM's MAPE on real held-out data? A curve below the $Y = 0$ baseline says yes. 5y/3y are S&P 500; the same 62.5/37.5 ratio is kept on GOOG and ZC=F.

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
- 8. TATR: synthetic-data protocols**
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

Three ways to produce the $Y \cdot 252$ synthetic days

Same FTS-Diffusion, three Markov-chain rollouts

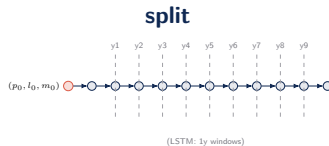


... (10 chains total)

Re-init every year from the same (p_0, l_0, m_0) .

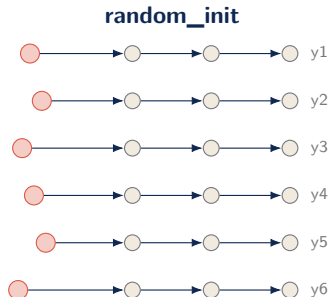
Pro: paper-faithful.

Con: no cross-year MC.



Single continuous MC; year cuts only affect the LSTM rolling window.

Same budget; MC persists across years.



... (10 chains total)

Re-init each year; init state $\sim$ SEGMENTS_INIT.

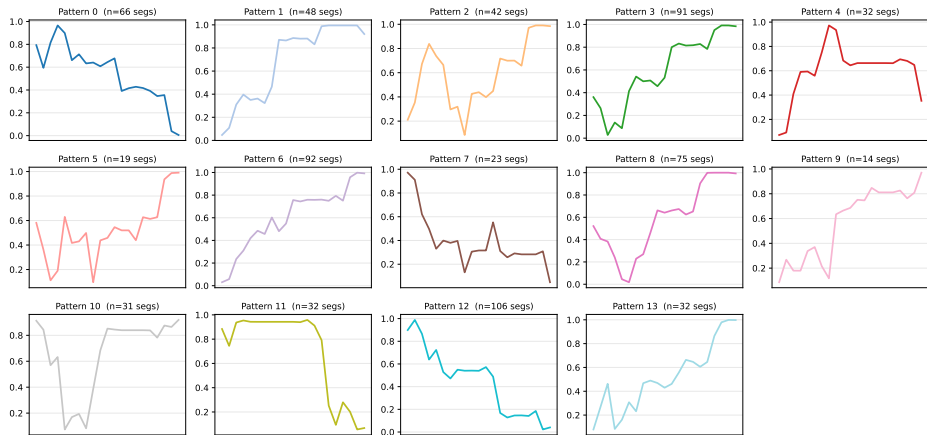
Diagnostic: isolates init from MC.

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
- 9. SISC: patterns on real data**
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

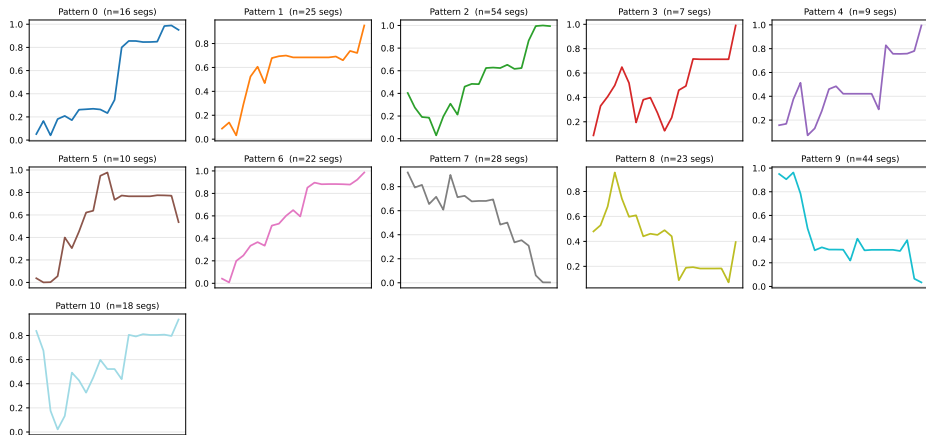
S&P 500: learned motifs ($K = 14$)

SISC patterns learned for S&P 500 ($K=14$)



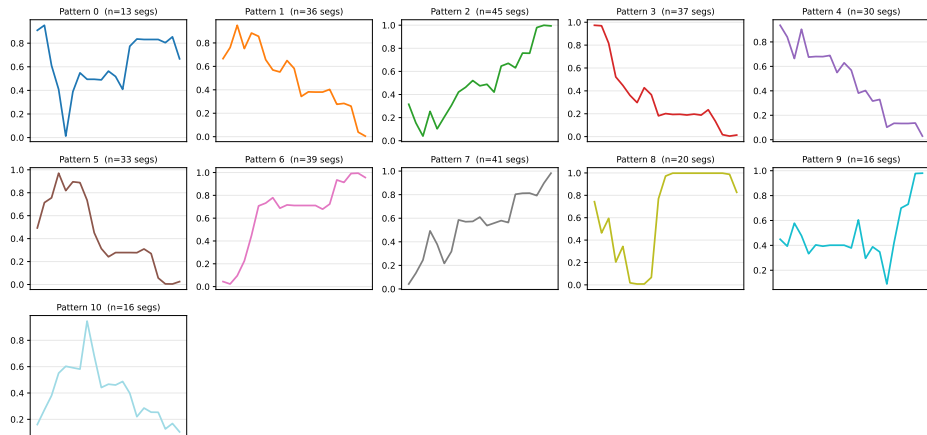
GOOG: learned motifs ($K = 11$)

SISC patterns learned for GOOG ($K=11$)



ZC=F (corn futures): learned motifs ($K = 11$)

SISC patterns learned for ZC=F (Corn futures) ($K=11$)

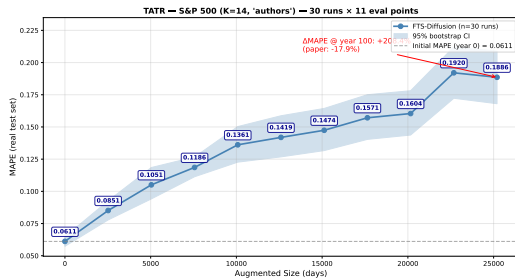


Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
- 10. TATR results: authors vs split**
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

S&P 500, $K = 14$: authors vs split

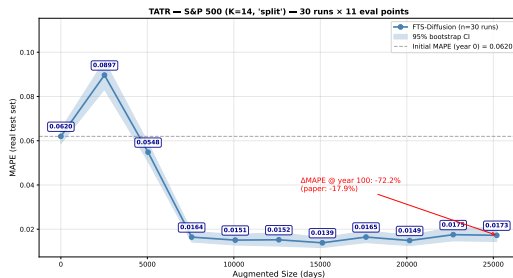
authors



MAPE delta: +208.47% vs. baseline.

Same data budget; only the MC connectivity changes between protocols.

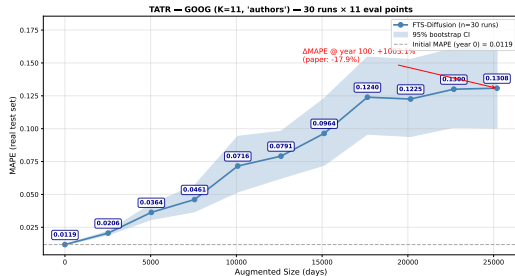
split



Split: continuous chain across years.

GOOG, $K = 11$: authors vs split

authors



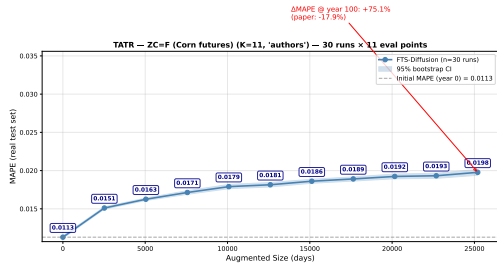
split (TODO: insert split panel, figure pending)

*per-run CSVs available
under results/tatr/goog/k11/split/
final plot not yet generated
(figure pending)*

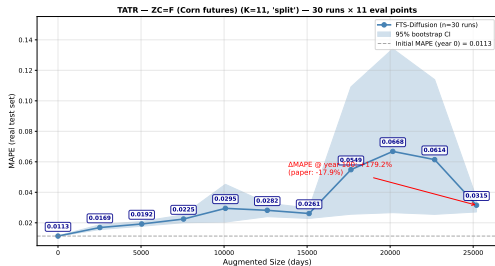
$K = 11$ on GOOG: smaller post-SISC budget shrinks the cluster count proportionally.

ZC=F, $K = 11$: authors vs split

authors

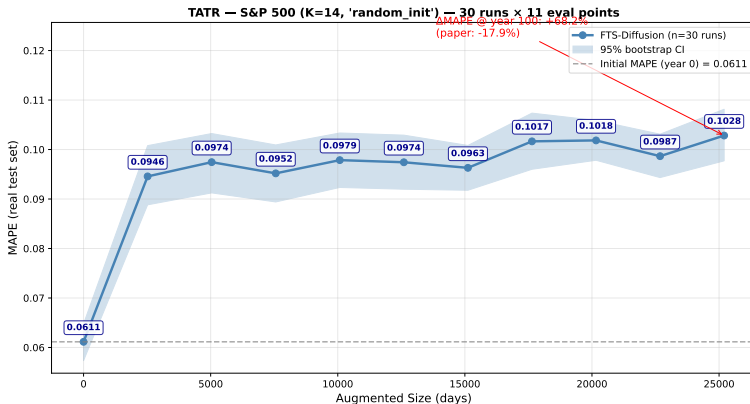


split



Same $K = 11$ as GOOG; the MAPE level differs because the motif library does.

S&P 500, $K = 14$: diagnostic, random init

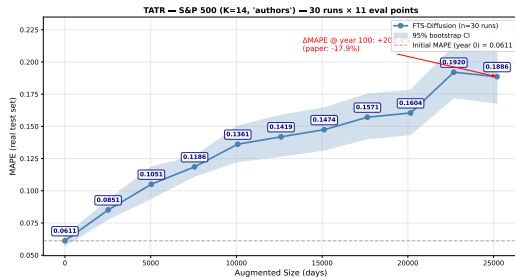


Random init isolates the initialisation effect from the MC-evolution effect.

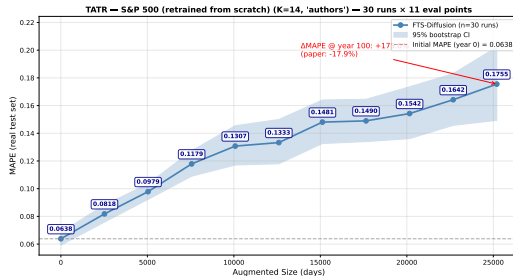
Authors' checkpoint vs. trained-from-scratch (S&P 500, $K = 14$)

Same authors protocol; only the PGM+PEM weights differ

authors' checkpoint



trained from scratch



Why $K = 14$ here: keeps the authors' data budget on S&P 500; on the smaller assets we use $K = 11$ because the post-SISC training set shrinks proportionally.

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
- 11. Further experiments**
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

TMTR: Train More, Test Real

Status

Experiment in progress. Figures will be inserted here once the runs complete.

figure slot: TMTR per-asset summary

5-day-ahead forecasting

Status

Experiment in progress. No numerical results to report yet.

figure slot: 5-day-ahead MAPE / hit-rate

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
- 12. Methodological observations**
13. Methodological observations
14. Limitations and Extensions
15. Conclusions

Reproducibility issues uncovered

- **Hard-coded LSTM warm-up.** `init_period = 252*5` in the downstream LSTM assumes 5 years of trading days. Breaks silently for GOOG and ZC=F (insufficient history); both run but evaluate the LSTM in its warm-up regime.
- **Device mismatch in `train_evolution_module`.** Target tensors are not moved to GPU. Forward stays on CPU even with a CUDA model; training is 8–10× slower and the optimiser trajectory differs.
- ***x*-range multiplier in `plot_downstream_tatr`.** Augmentation axis multiplied by 10 in plotting; mis-labels “10×” as “100×”.
- **Under-specified synthetic-data protocol** — the paper does not pin down “authors” vs. continuous-chain rollout (split).

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
- 13. Methodological observations**
14. Limitations and Extensions
15. Conclusions

Why exact reproduction was difficult

- **Synthetic-data protocol is under-specified.** The paper does not fully specify whether synthetic paths are generated as independent segments, one continuous chain, or a continuous chain later split into evaluation blocks.
- **Downstream split is not portable across assets.** The released downstream LSTM uses `init_period = 252*5`. This is compatible with the S&P 500 setup, but GOOG and ZC=F do not have enough held-out history under the same split. Any valid GOOG/ZC=F run therefore requires an adapted protocol.
- **GPU training path requires a patch.** In `train_evolution_module`, target tensors are not moved to the model device. On CUDA this creates a device mismatch, so the intended GPU path is not directly reproducible without modifying the code.
- **Published trends are sensitive to protocol choices.** Our experiments show that changing the synthetic rollout protocol can materially change the TATR trend, even with the same assets and similar hyperparameters.

Markov-chain stationarity

Setup

PEM defines a Markov chain over $s_m = (p_m, \alpha_m, \beta_m)$. Stationarity would require a fixed invariant π with $\pi^\top T = \pi^\top$.

- Empirical T (per asset, per K) is right-stochastic; principal left-eigenvector gives π .
- Stationarity check: simulate long chains, monitor $\|\hat{\pi}_t - \pi\|_1$ over time.
- Drift check: compare mean motif occupancy in real vs. synthetic across the chain.

Open question — empirical burn-in

Under the `split` protocol, no clear stationarity is reached within one simulated year. How many segments of burn-in are needed before the chain forgets its init state? Not answered here.

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
- 14. Limitations and Extensions**
15. Conclusions

Limitation 1: motif discovery is fragile

The whole generator depends on the first segmentation step

Discrete motif extraction

SISC turns a continuous time series into a finite dictionary of motifs. This is powerful, but it creates several sensitivities.

- The number of clusters K and length range $[l_{\min}, l_{\max}]$ are important hyperparameters.
- Greedy segmentation can misplace boundaries between adjacent motifs.
- DTW captures shape similarity, but may ignore economically meaningful differences.
- Motifs are learned offline; new market regimes may require re-clustering.

Implication: generation quality is only as good as the learned motif vocabulary.

Limitation 2: local patterns may miss long-range structure

Short motifs are not the same as full market dynamics

Captured well

- Local shape recurrence
- Heavy-tailed return behavior
- Volatility clustering proxies
- Segment-level scaling

Harder to capture

- Long-memory dependence
- Regime persistence
- Macro or news conditioning
- Rare crises and structural breaks
- Cross-asset co-movement

The learned transition is essentially a compressed state evolution:

$$(p_m, \alpha_m, \beta_m) \mapsto (p_{m+1}, \alpha_{m+1}, \beta_{m+1}).$$

Limitation 3: realism does not imply no artificial alpha

Stylized facts are necessary but not sufficient

The paper evaluates whether synthetic series resemble real data through distributional tests, stylized facts, and downstream prediction experiments.

Remaining concern

A synthetic generator can match heavy tails and volatility clustering, while still introducing **systematic predictable drift** under a simple trading signal.

- Good marginal fit does not guarantee correct conditional dynamics.
- Prediction experiments may depend on the model architecture used for evaluation.
- Leakage is possible if motif extraction or scaling is fitted using future data.
- A generator useful for augmentation should not create spurious predictability.

Why no predictable drift matters

Martingale intuition for financial returns

A standard benchmark assumption in financial market modelling is that returns should not contain an easily exploitable conditional mean under the available information:

$$\mathbb{E}[r_{t+1} \mid \mathcal{F}_t] = 0,$$

or, equivalently, for every admissible test function $g(\mathcal{F}_t)$,

$$\mathbb{E}[r_{t+1}g(\mathcal{F}_t)] = 0 = (\text{Cov}(r_{t+1}, g(\mathcal{F}_t)) + \mathbb{E}[r_{t+1}] \mathbb{E}[g(\mathcal{F}_t)]) ..$$

Financial interpretation

Given the information available at time t , the next return should not be systematically predictable. Otherwise, the generated data may contain **artificial alpha**.

- Real financial returns are noisy and close to unpredictable at short horizons.
- A good synthetic generator should match stylized facts without creating fake trading signals.
- Therefore, distributional realism alone is not enough: we also need a conditional-mean diagnostic.

Extension 1: no-predictable-drift diagnostic

A fixed test for artificial conditional mean

Use a simple fixed information set:

$$\omega_t = (1, r_t, r_{t-1}, |r_t|, |r_{t-1}|)^\top.$$

Define the drift-violation statistic:

$$\delta = \sup_{\|b\|_2 \leq 1} \left| \widehat{\mathbb{E}} [r_{t+1} b^\top \omega_t] \right|.$$

For this linear class:

$$\delta = \left\| \widehat{\mathbb{E}} [r_{t+1} \omega_t] \right\|_2.$$

Compare three objects

$$\delta_{\text{real}}, \quad \delta_{\text{synthetic}}, \quad \delta_{\text{null}}.$$

The null can be built by block-shuffling returns to preserve rough marginal behavior while breaking predictability.

Extension 2: drift-regularized generation

Keep stylized facts, discourage artificial alpha

Modify training with one additional penalty:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{FTS}} + \lambda \delta_{\text{synthetic}}.$$

- $\mathcal{L}_{\text{FTS}}$: original reconstruction, diffusion, and transition losses.
- $\delta_{\text{synthetic}}$: fixed no-predictable-drift statistic.
- λ : chosen once on validation data.

Evaluation question

Does the penalty reduce predictable drift without destroying:

- heavy-tailed returns;
- volatility clustering;
- distributional similarity to real data?

Extension 3: broader model directions

Multivariate FTS-Diffusion

Generate vector returns:

$$r_t \in \mathbb{R}^d$$

and preserve cross-correlations, co-jumps, and sector-level dependence.

Regime-conditioned transitions

Condition the transition module on market state:

$$\phi(p_m, \alpha_m, \beta_m, c_m).$$

Stress-aware generation

Oversample rare but realistic regimes: crises, crashes, liquidity shocks, and volatility spikes.

Stronger validation

Move beyond distribution matching:

- martingale-style diagnostics;
- trading-rule robustness;
- risk metrics such as VaR and ES;
- out-of-sample transfer to new assets.

Takeaway

Main limitation

FTS-Diffusion is strong at generating realistic local motif-based financial paths, but its guarantees are mostly empirical and distributional.

Main extension

Add a fixed no-predictable-drift diagnostic and, optionally, a drift penalty:

realistic synthetic data + no artificial predictability.

This gives a cleaner benchmark for deciding whether synthetic financial time series are useful for augmentation rather than merely visually or marginally realistic.

Conclusion

- FTS-Diffusion addresses the core challenges of irregularity and scale-invariance in financial data.
- SISC allows for clustering recurring market “shapes” effectively.
- The model demonstrates superior performance in preserving statistical properties compared to TimeGAN and QuantGAN.

Outline

1. Motivation
2. Architecture overview
3. SISC: three animated steps
4. Generation module (PGM)
5. Evolution module (PEM)
6. Training data
7. TATR: the downstream benchmark
8. TATR: synthetic-data protocols
9. SISC: patterns on real data
10. TATR results: authors vs split
11. Further experiments
12. Methodological observations
13. Methodological observations
14. Limitations and Extensions
- 15. Conclusions**

Contributions

- Faithful from-scratch replication of FTS-Diffusion (SISC + PGM + PEM) on three assets: S&P 500, GOOG, ZC=F.
- Identified four reproducibility issues in the public reference implementation; documented patches.
- Defined and compared three synthetic-data protocols (authors / split / random_init); showed the headline trend depends on the protocol.
- Added per-run MAPE confidence intervals and a paper-vs.-replication gap diagnostic.
- Pattern dictionaries learned and visualised for $K \in \{11, 13, 14\}$ across all three assets, plus MC transition-state figures.

Open questions

- Which protocol *should* be the benchmark? `split` is the only one that exploits cross-year MC evolution, but stationarity is not guaranteed.
- Is the PEM Markov chain stationary in practice, and on what timescale?
- Why does MAPE grow with augmentation under the authors' protocol: leakage from the rolling-window scheme, or genuine distributional drift?
- How sensitive are downstream results to K ? Qualitative flips on $ZC=F$ between $K=11$ and $K=13$.

Bottom line

The architecture is sound; the experimental protocol is not. A clean replication requires both.

Backup: repository layout

```
ml-in-fcs/  
  architectures/{asset}/k{K}/{data,res,trained_models}/    <- SISC + PGM + PEM ckpts  
  synthetic/{asset}/k{K}/{protocol}/run_XX_{syn,states}.npz  
  results/tatr/{asset}/k{K}/{protocol}/run_XX.csv           <- per-run MAPE  
  figures/{sisc,tatr,mc_analysis}/{asset}/k{K}/...          <- final PDFs  
  src/{TATR_Reduced_Replication.ipynb,  
        FTS_Diffusion_Training.ipynb,  
        B3_SISC_Simulated_Investigation.ipynb}
```

Deli Lin, Eric Shao, Lapo Linossi, Tom Haidinger

ETH Zurich

Machine Learning in Finance and Complex Systems

Supervisor: Alexander Arandjelovic

Thank you. *Questions?*